

Ahmad Abdel Razak Hussein Alkaseb
Benjamin Sebastian Barrales Hernandez
Jeppe Rønning Koch
Laith Abdel Razak Hussein Alkaseb

Exam Project Description

Astralis Backend — Testing

Project Overview

Astralis is a Java-based REST API backend for an internal admin panel built for an esports organisation. The system replaces an unstructured Google Drive setup and provides secure, structured management of accounts, games, licenses, to-do tasks, Q&A entries, and content (headers, footers, information posts) with role-based access control.

1. Review

One group member will author an SRS (Software Requirements Specification) for the Astralis system. The chosen format is a series of user stories covering the core domain entities: accounts, roles, games, licenses, to-do tasks, Q&A entries, and content management.

The remaining members will conduct a formal review of the SRS and produce a structured review document, noting defects and improvement suggestions.

Deliverables
SRS – Software Requirements Specification (PDF) — descriptive document covering all functional areas
SRS Review (PDF) — formal review findings and sign-off

2. Risk Assessment

A risk assessment will be performed throughout the project lifecycle. Risks are identified for the Astralis system across technical, organisational, and testing dimensions.

Risk tables will be produced at multiple stages:

- Initial table — identified at project start
- Mid-development table — updated after core implementation
- Final table — reflecting resolved and residual risks

Risk tables to visualise likelihood vs. impact at each phase.

Deliverables
Risk tables (initial, mid-development, final) — PDF
Risk matrices illustrating risk state at each phase — PDF

3. Black-Box Test Design

Black-box techniques are applied to the Astralis REST API endpoints and domain logic without knowledge of internal implementation. The following techniques are applied:

Equivalence Partitioning

Input fields across the API are partitioned into valid and invalid equivalence classes. Examples include:

- Username length: too short / valid range / too long
- Password: below minimum length / valid / exceeds maximum
- License keys: valid format / invalid format / empty
- Role assignment: valid role / nonexistent role / null

Boundary Value Analysis

Boundary values are tested at the edges of each equivalence class — for example, the exact minimum and maximum character counts for username and password fields, as enforced by the @Size Bean Validation constraints.

Deliverables

Equivalence partitioning analysis and test cases — PDF

Boundary value analysis and test cases — PDF

4. Static Testing Tools & White-Box Test Design

Static analysis tools are used to assess code quality and guide test design. The project integrates SonarQube via GitHub Actions CI for continuous static analysis, reporting code bugs, vulnerabilities, and duplication.

Code coverage is measured using JaCoCo generating a report during the mvn verify phase. Coverage data is used to identify untested code paths and inform the design of additional unit tests.

Deliverables

SonarQube report screenshots and explanation of findings — PDF

JaCoCo code coverage report and explanation of how it guided unit test design — PDF

5. Unit Testing & Integration Testing

The Astralis test suite has two clearly separated layers in `src/test/java/`:

Layer	Package	Scope
Unit tests	<code>unit/</code>	Unit tests for business logic
Integration tests	<code>integration/</code>	Integration tests on DB access objects directly

Key design decisions:

- All tests run against a real PostgreSQL database via Testcontainers (no mocks for the DB layer)
- `@BeforeEach` seeds consistent test data; `@AfterEach` truncates all tables with `RESTART IDENTITY CASCADE`
- Black-box test cases from phase 3 are implemented here as parameterised JUnit 5 tests

- Parameterised tests / data providers are used throughout for equivalence classes and boundary values
- Comprehensive assertions on both success and error paths

Deliverables
Source code of the Astralis application
Source code of unit tests and integration tests (src/test/java/)

6. Continuous Testing

The project uses GitHub Actions for continuous integration, triggered on every push and pull request to main. The CI pipeline automates:

- Compilation and build via Maven
- Execution of all DAO and controller integration tests
- API tests
- JaCoCo coverage report generation
- SonarQube static analysis (when secrets are configured)

Deliverables
CI pipeline script — workflow.yml (GitHub Actions YAML)
CI pipeline output / screenshots — PDF

7. API Testing

All REST endpoints of the Astralis API are tested. Tests cover:

- HTTP status codes for all endpoints (200, 201, 400, 401, 403, 404, etc.)
- Response body validation — JSON structure and field values
- Execution time / response time assertions
- Positive tests — valid inputs yielding expected results
- Negative tests — invalid inputs, missing auth tokens, wrong roles, malformed payloads

Endpoints covered span all controllers: Account, Game, License, Todo, QA, Information, Role, Header, Footer.

Deliverables
API test collection and environment — JSON format

8. End-to-End UI Testing

A suite of end-to-end tests is implemented in code using a UI automation framework (Playwright) against the Astralis frontend.

Tests cover key user journeys through the admin panel, such as:

- Login flow
- CRUD operations on games, licenses, and to-do tasks through the UI
- Role-based access control verification (admin vs. user views)
- Validation feedback for invalid form inputs

Deliverables
Source code of end-to-end UI tests

9. Stress & Performance Testing

Load, stress, and spike testing is performed on the Astralis backend API using a performance testing tool. Scenarios include:

- Load test — sustained normal traffic across multiple endpoints simultaneously
- Stress test — traffic gradually increased beyond expected peak to find the breaking point
- Spike test — sudden large burst of requests to simulate unexpected traffic spikes

Metrics captured include response, error rates, throughput, and server resource utilisation under each scenario.

Deliverables

Performance test reports or screenshots documenting load, stress, and spike test results